

AutoDiCE in Docker

Running the distributed AlexNet example — command guide

AutoDiCE splits a neural network across several compute nodes and runs the pieces in parallel, passing intermediate results between them over MPI. This guide covers every command you need to build the image, run the bundled AlexNet example, point it at your own model, and control which machines the ranks land on.

Everything runs inside the container. Docker is the only thing you install.

The short version

Build it once, then run it with your models folder mounted. Models are never baked into the image, so one image serves any model you give it.

```
docker build -t autodice .
docker run --rm -v $L/models:$V/models:ro -v $L/out:$V/out autodice
```

What the example does	
Model	BVLC AlexNet (bvlc_alexnet-9.onnx), 24 layers
Split point	After fc6_2 — 18 layers on rank 0, 6 on rank 1
Transferred	One tensor, fc6_2, sent rank 0 → rank 1
Input	dog.jpg, a retriever photo
Output	Top-3 ImageNet classes

1. Quick start

Run everything from the repository root.

Fetch the model

The ONNX weights are not in the repository, so this step is not optional on a fresh clone. About 233 MB.

```
mkdir -p $L/models
curl -L -o $L/models/bvlc_alexnet-9.onnx \
  https://github.com/onnx/models/raw/main/validated/vision/\
classification/alexnet/model/bvlc_alexnet-9.onnx
```

The URL is split across lines here to fit the page — join the last two parts into one address when you paste it.

Build the image

Once, and again only when you change the source. It compiles the whole ncnn library, so expect a few minutes.

```
cd /home/gkorod/AutoDiCE
docker build -t autodice .
```

Run the example

The paths get long, so these two shorthands are used throughout this guide. Paste them into your shell first, or write the paths out in full.

```
L=$(pwd)/tools/distributed/vertical      # on your machine
V=/autodice/tools/distributed/vertical    # inside the container
```

```
docker run --rm -v $L/models:$V/models:ro -v $L/out:$V/out autodice
```

Two folders, one direction each

`models/` holds your networks and is only ever read — mount it read-only. Everything the pipeline generates goes to `out/`. The `models` mount is required: the image contains no networks at all, so without it the run stops with “No model found”. The `out` mount is optional — leave it off and the results are simply discarded with the container.

The last lines you should see:

```
run distributed inference
215 = 0.589278
207 = 0.127041
213 = 0.103414
Brittany spaniel
golden retriever
Irish setter, red setter
```

That is AlexNet classifying the bundled dog photo, with the network split across two MPI processes.

--rm deletes the container when it exits. Whatever the run generates survives only in the `out/` mount.

2. What happens when you run it

The container's default command is `run . sh`, which performs four steps in order:

1	Split the model interface.py reads mapping.json, cuts the ONNX graph into one sub-model per node, and generates a matching multinode.cpp.
2	Convert to ncnn onnx2ncnn turns each sub-model into the .param + .bin pair the inference engine loads.
3	Compile multinode.cpp is built into the ./multinode binary, linked against ncnn and MPI.
4	Run distributed inference mpirun launches one process per rank. Rank 0 runs the convolutions and sends fc6_2 to rank 1, which finishes the fully-connected layers and prints the top-3.

3. Every command, by situation

All of these assume the \$L and \$V shorthands from section 1.

Run the complete example

The normal case. The sub-models, the ncnn weights, the rankfile, the generated multinode.cpp and the compiled binary all land in out/; models/ is never written to.

```
docker run --rm -v $L/models:$V/models:ro -v $L/out:$V/out autodice
```

Files in out/ are created by root inside the container, so they land on your disk owned by root.

One model in the folder — nothing to configure

If the models folder holds exactly one network, that is the one that gets split. Nothing generated is ever written there, so this stays unambiguous no matter how many times you run.

\$L/models/	\$L/out/
vgg16-7.onnx	lenovo_cpu0.{onnx,param,bin}
	lenovo_cpu1.{onnx,param,bin}
	rankfile, hostfile, sender.json, receiver.json
	multinode.cpp, multinode

Several models in the folder

The folder can hold as many networks as you like, but then the choice is ambiguous and you have to name one with AUTODICE_MODEL, as a path relative to the example directory:

```
docker run --rm -v $L/models:$V/models:ro -v $L/out:$V/out \  
-e AUTODICE_MODEL=models/vgg16-7.onnx autodice
```

Without it the run stops and lists what it found, rather than guessing:

```
Several models found in ./models/: bvlc_alexnet-9.onnx, vgg16-7.onnx  
Pick one with AUTODICE_MODEL, e.g.  
docker run --rm -e AUTODICE_MODEL=./models/bvlc_alexnet-9.onnx ... autodice
```

Remember that mapping.json has to match whichever model you select — the layer names differ per network. See section 5.

Use a models folder from somewhere else entirely

Nothing ties the volume to the repository. Point it at any directory.

```
docker run --rm -v /data/my-onnx-zoo:$V/models:ro -v $L/out:$V/out \  
-e AUTODICE_MODEL=models/mymodel.onnx autodice
```

Do you need to clean up between runs?

No. There is no container to stop — --rm deletes it the moment run.sh finishes — and there is no Docker volume to remove either, because the mount is a bind mount of a folder on your disk, not a named volume. `docker volume ls` will not show it.

Nothing accumulates in the models folder, because nothing is ever written there. Everything in out/ is regenerated and overwritten on each run, so you can leave it as it is — including when you rename node keys in mapping.json or switch to a different network.

If you want a guaranteed-fresh result anyway, delete the output folder; it is recreated automatically:

```
rm -rf tools/distributed/vertical/out
```

Sub-models from a previous mapping do stay in out/ under their old names until you do that. They are never loaded — the run only reads the names currently in mapping.json — so they waste disk and nothing else.

Split the model only, without running inference

Useful when you just want to inspect the partition.

```
docker run --rm -v $L/models:$V/models:ro -v $L/out:$V/out \
  autodice python3 interface.py
```

Work inside the container

Drops you in a shell at the example directory.

```
docker run --rm -it -v $L/models:$V/models:ro -v $L/out:$V/out autodice bash
```

From there:

```
bash run.sh                # the full pipeline

cd out                     # re-run inference only
mpirun --allow-run-as-root --oversubscribe \
  -np 2 -rf rankfile ./multinode dog.jpg
```

Classify a different image

```
docker run --rm -v /path/to/cat.jpg:$V/dog.jpg autodice
```

4. Choosing which machine each rank runs on

Usually you can skip this section

By default the rankfile is written with the container's own hostname, so a single-machine run needs no configuration at all. You only need these settings when the ranks must run on different machines.

Where the names come from

Each key in `mapping.json` has the form `<device>_<resource>`:

```
lenovo_cpu0
|_____|_|
|      +-- cores to pin to      -> "slots=0" in the rankfile
+----- logical device name -> which host MPI should use
```

The device part is only a label. Two environment variables decide what real host it turns into:

Variable	What it does
AUTODICE_HOST	One host for every rank. Defaults to the machine's own hostname.
AUTODICE_HOSTS	Per-device, comma-separated device=host pairs. Devices not listed fall back to AUTODICE_HOST.

One device, default — nothing to set

```
docker run --rm autodice
```

One device, a specific name

Both flags are needed and must match: the environment variable decides what goes into the rankfile, and `--hostname` makes the container actually answer to that name.

```
docker run --rm --hostname lenovo -e AUTODICE_HOST=lenovo autodice
```

Set only the environment variable and the rankfile names a host the container is not. It still runs and still gives the right answer, but prints “not in rankfile, running without core pinning” and you lose CPU pinning.

Several devices, one host each

With mapping keys `alpha_cpu0` and `beta_cpu1`:

```
docker run --rm -e AUTODICE_HOSTS="alpha=node-a,beta=node-b" \
  autodice python3 interface.py
```

produces:

```
--- rankfile ---          --- hostfile ---
rank 0=node-a  slots=0    node-a
rank 1=node-b  slots=1    node-b
```

Real multi-machine runs need more than names

Setting the hosts tells mpirun where to launch, but getting there also requires passwordless SSH between the machines (OpenMPI launches remote ranks over SSH, and this image ships no SSH daemon), the models/ directory present on every host since each rank loads its own weights from disk, and the MPI port range open between them.

5. Using your own model

Nothing in the pipeline is AlexNet-specific. Four steps.

1	Put your .onnx file in models/ Alongside the bundled AlexNet.
2	Get the layer names Not the original ONNX node names — see the warning below.
3	Write mapping.json Split the layer list, in order, into one group per node.
4	Select it with AUTODICE_MODEL No code change needed — the model is chosen at run time.

Step 2 in detail

The names are rewritten before splitting

AutoDiCE renames every node to its first output tensor, with slashes turned into underscores. So the names in mapping.json will not match what you see in Netron. Always dump them from the formatted model with the command below.

```
docker run --rm -v $L/models:$V/models:ro autodice python3 -c "  
from onnx_split import format_onnx  
import onnx  
m = onnx.load(format_onnx('models/YOURMODEL.onnx'))  
print([n.name for n in m.graph.node])"
```

For the bundled AlexNet this prints the same 24 names that appear in mapping.json:

```
['conv1_1', 'conv1_2', 'norm1_1', 'pool1_1', ... ,  
'fc6_2', 'fc6_3', 'fc7_1', ... , 'prob_1']
```

Step 3 in detail

```
{  
  "lenovo_cpu0": ["conv1_1", "...", "fc6_2"],  
  "lenovo_cpu1": ["fc6_3", "...", "prob_1"]  
}
```

Every layer must appear exactly once. The tool checks this and prints `Consistency Check Pass`. — if you miss any, it lists them for you. You are not limited to two nodes: add more keys and the rankfile, the sends and receives, and the generated C++ all scale to match.

Split where the tensor is small. The intermediate blob crosses the network on every inference, so a fully-connected boundary like fc6_2 costs far less than a cut in the middle of the convolutional stack.

Iterating without rebuilding

Mount the files you are editing instead of rebuilding the image each time.


```
docker run --rm \
-v $L/mapping.json:$V/mapping.json \
-v $L/interface.py:$V/interface.py \
-v $L/models:$V/models:ro \
-v $L/out:$V/out \
autodice
```

Your model also has to survive onnx2ncnn. If it uses operators ncnn does not implement, conversion prints unsupported-operator warnings and the run fails. That is an ncnn limitation. AlexNet, VGG and DenseNet are the models this repository has been exercised on; ResNet needed a fix for its batch dimension.

6. If something goes wrong

What you see	What it means	What to do
not in rankfile, running without core pinning	The rankfile names a host this machine is not.	Harmless — results are still correct, only CPU pinning is lost. To fix, add <code>--hostname</code> matching your <code>AUTODICE_HOST</code> .
mpirun has detected an attempt to run as root	OpenMPI refuses to run as root by default.	Add <code>--allow-run-as-root</code> . <code>run.sh</code> already does.
lack of authority to execute on one or more specified nodes	mpirun cannot reach a host named in the rankfile.	The hostname does not resolve, or SSH is not set up. See section 4.
No model found in ./models/	The image has no models in it; the volume is missing or empty.	Add <code>-v \$L/models:\$V/models:ro</code> and put a .onnx file in it.
Several models found in ./models/	More than one network in the folder, so the choice is ambiguous.	Name one with <code>AUTODICE_MODEL</code> , or keep a single model per folder.
failed to bind memory	Containers cannot bind memory to a NUMA node.	Expected and harmless — CPU pinning still applies. The only way to silence it is <code>--bind-to none</code> , which discards the pinning.
Consistency Check Fail.	mapping.json does not cover the model exactly.	The message lists the missing or extra layers. Every layer, exactly once.
Unsupported operator warnings from onnx2ncnn	ncnn has no implementation for an operator in your model.	Use a different model, or replace the operator before exporting.

7. Every environment variable

All optional — the defaults are exactly what the quick start uses.

Variable	Default	Purpose
AUTODICE_MODEL	auto-detect	Which network in models/ to split. Only needed when more than one is there.
AUTODICE_IN	./models	Input directory inside the container.
AUTODICE_OUT	./out	Output directory inside the container.
AUTODICE_HOST	local hostname	One MPI host for every rank.
AUTODICE_HOSTS	—	Per-device MPI hosts, e.g. <code>lenovo=node-a, jetson=node-b</code> .

8. Where things live

Path	Purpose
Dockerfile	Repository root. Ubuntu 22.04, ncnn + MPI + OpenMP, CPU only.
.dockerignore	Keeps models/ out of the build context, so the image ships no weights.

Path	Purpose
<code>tools/distributed/vertical/</code>	The example. The container starts here.
<code>run.sh</code>	The four-step pipeline. The image's default command.
<code>interface.py</code>	Splits the model, generates the C++, writes the rankfile.
<code>mapping.json</code>	Which layers go to which node. Edit to change the split.
<code>models/</code>	INPUT. Your ONNX networks. Not in the image — mount it read-only.
<code>out/</code>	OUTPUT. Everything a run generates. Safe to delete at any time.

The compiled binary contains no hostnames at all — it dispatches on the MPI rank number. Only `models/rankfile` and `models/hostfile` are host-specific, and you can edit them by hand after splitting if that is easier than setting the variables.